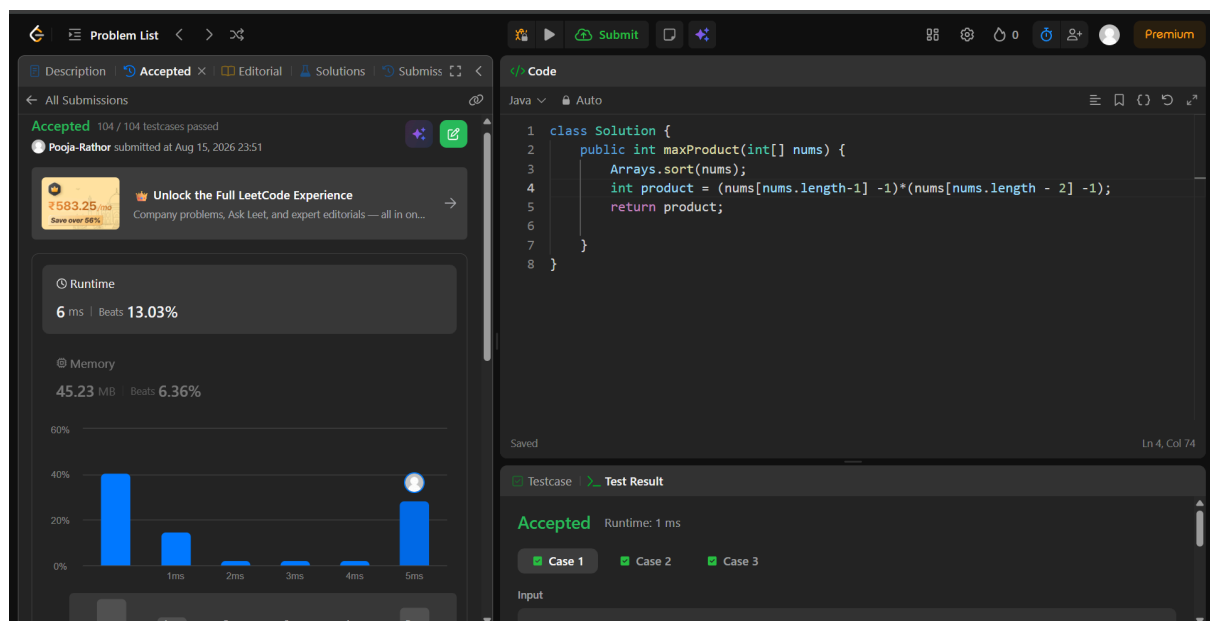


Name - Pooja Rathor
University Roll No. - 12515001119
Class Roll No. - 34
Section - CB

1464. Maximum Product of Two Elements in an Array

```
class Solution {  
    public int maxProduct(int[] nums) {  
        Arrays.sort(nums);  
        int product = (nums[nums.length-1]  
-1)*(nums[nums.length - 2] -1);  
        return product;  
    }  
}
```



Time Complexity - $O(n \log n)$

Second-Largest

```
class Solution {  
    public int getSecondLargest(int[] arr) {  
        Arrays.sort(arr);  
        for(int i = arr.length - 1; i > 0; i--){  
            if (arr[i] != arr[i-1]){  
                return arr[i-1];  
            }  
        }  
        return -1;  
    }  
}
```

im

```
    }  
}
```

The screenshot displays a coding platform interface with a dark theme. On the left, the 'Output Window' shows 'Compilation Results' for 'Custom Input' by 'Y.O.G.I. (AI Bot)'. A green checkmark indicates 'Problem Solved Successfully'. Below this, statistics are shown: 'Test Cases Passed: 1120 / 1120', 'Attempts: Correct / Total: 1 / 6', 'Accuracy: 16%', 'Points Scored: 2 / 2', and 'Your Total Score: 3'. On the right, the code editor shows the Java solution for the 'Second Largest' problem, which sorts the array and returns the second largest element. The code is as follows:

```
1 class Solution {  
2     public int getSecondLargest(int[] arr) {  
3         Arrays.sort(arr);  
4         for(int i = arr.length - 1; i > 0; i--){  
5             if (arr[i] != arr[i-1]){  
6                 return arr[i-1];  
7             }  
8         }  
9         return -1;  
10    }  
11 }  
12  
13 }
```

At the bottom right, there are buttons for 'Custom Input', 'Compile & Run', and 'Submit'.

Time Complexity - $O(n \log n)$

162. Find Peak Element

```
class Solution {
    public int findPeakElement(int[] nums) {
        for(int i = 0; i < nums.length-1; i++){
            if (nums[i] > nums[i + 1]){
                return i;
            }
        }
        return nums.length - 1;
    }
}
```

The screenshot displays the LeetCode submission interface for the 'Find Peak Element' problem. The code is shown in the 'Code' tab, and the 'Test Result' section indicates that the solution is 'Accepted' for both Case 1 and Case 2. The runtime is 0 ms, and the memory usage is 44.12 MB.

Runtime: 0 ms | Beats 100.00%

Memory: 44.12 MB | Beats 57.17%

Test Result: Accepted Runtime: 0 ms

Case 1 **Case 2**

Input

Time Complexity - $O(n)$

33. Search in Rotated Sorted Array

```
class Solution {  
    public int search(int[] nums, int target) {  
        for(int i = 0; i < nums.length; i++){  
            if(nums[i]== target){  
                return i ;  
            }  
        }  
        return -1;  
    }  
}
```

The screenshot displays the LeetCode submission interface for the problem "33. Search in Rotated Sorted Array". The submission is marked as "Accepted" with 197/197 testcases passed. The user, Pooja-Rathor, submitted the solution on August 16, 2026, at 21:18. The runtime is 0 ms, and the memory usage is 43.87 MB, which beats 47.24% of other submissions. The code is shown in a dark-themed editor, and the test results show "Accepted" for all three cases.

Accepted 197 / 197 testcases passed
Pooja-Rathor submitted at Aug 16, 2026 21:18

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in on...

Runtime
0 ms | Beats 100.00%

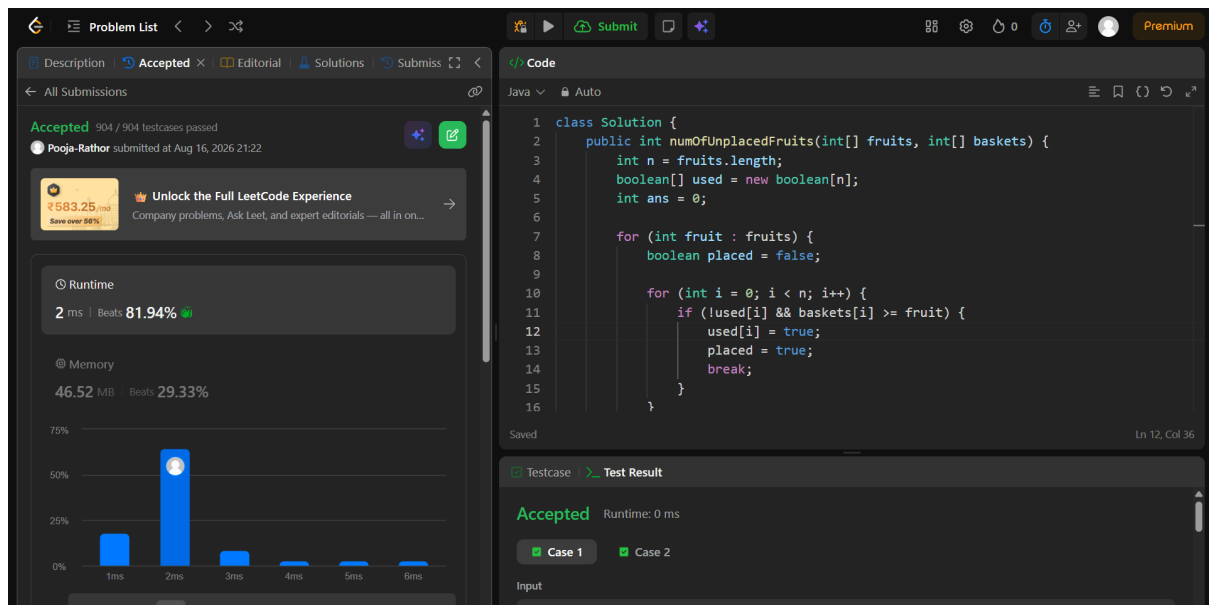
Memory
43.87 MB | Beats 47.24%

Testcase Test Result
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3

Time Complexity - $O(n)$

3477. Fruits Into Baskets II

```
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[]
baskets) {
        int n = fruits.length;
        boolean[] used = new boolean[n];
        int ans = 0;
        for (int fruit : fruits) {
            boolean placed = false;
            for (int i = 0; i < n; i++) {
                if (!used[i] && baskets[i] >= fruit) {
                    used[i] = true;
                    placed = true;
                    break;
                }
            }
            if (!placed) {
                ans++;
            }
        }
        return ans;
    }
}
```



Time Complexity - $O(n^2)$

Best Time to Buy and Sell Stock

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = prices[0];
        int maxProfit = 0;
        for (int i = 1; i < prices.length; i++) {
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            }
            int profit = prices[i] - minPrice;

            if (profit > maxProfit) {
                maxProfit = profit;
            }
        }
        return maxProfit;
    }
}
```

The screenshot displays the LeetCode submission page for the problem "Best Time to Buy and Sell Stock". The submission is accepted, with 213/213 testcases passed. The user, Pooja-Rathor, submitted it on Aug 16, 2026 at 21:27. The runtime is 2ms, beating 42.73% of other solutions. The memory usage is 93.85 MB, beating 96.59% of other solutions. The code is written in Java and implements a single-pass algorithm to find the maximum profit.

```
1 class Solution {
2     public int maxProfit(int[] prices) {
3         int minPrice = prices[0];
4         int maxProfit = 0;
5
6         for (int i = 1; i < prices.length; i++) {
7             if (prices[i] < minPrice) {
8                 minPrice = prices[i];
9             }
10
11             int profit = prices[i] - minPrice;
12
13             if (profit > maxProfit) {
14                 maxProfit = profit;
15             }
16         }
17     }
18 }
```

Time Complexity - $O(n)$

1. Two Sum

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        int []arr = new int [2];
        for(int i = 0; i < nums.length; i++){
            for(int j = i +1; j < nums.length; j++){
                if((nums[i] + nums[j]) == target){
                    arr [0]= i;
                    arr [1] =j;
                }
            }
        }
        return arr;
    }
}
```

Accepted 65 / 65 testcases passed
Pooja-Rathor submitted at Aug 16, 2026 21:30

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in on...

Runtime: 49 ms | Beats 8.15%
Memory: 46.80 MB | Beats 90.95%

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        int []arr = new int [2];
        for(int i = 0; i < nums.length; i++){
            for(int j = i +1; j < nums.length; j++){
                if((nums[i] + nums[j]) == target){
                    arr [0]= i;
                    arr [1]=j;
                }
            }
        }
        return arr;
    }
}
```

Testcase Test Result
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3

Time Complexity - $O(n^2)$

1365. How Many Numbers Are Smaller Than the Current Number

```
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {
        int[] arr = new int[nums.length];
        for (int i = 0; i < nums.length; i++){
            int count = 0;
            for(int j = 0 ; j < nums.length; j++){
                if(nums[i] > nums[j] ){
                    count++;
                }
            }
            arr[i]=count;
        }
        return arr;
    }
}
```


Accepted 103 / 103 testcases passed
 Pooja-Rathor submitted at Aug 16, 2026 21:33

Runtime
 10 ms | Beats 16.23%

Memory
 45.34 MB | Beats 83.50%

```

1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3         int[] arr = new int[nums.length];
4         for (int i = 0; i < nums.length; i++){
5             int count = 0;
6             for(int j = 0 ; j < nums.length; j++){
7                 if(nums[i] > nums[j]){
8                     count++;
9                 }
10                arr[i]=count;
11            }
12        }
13        return arr;
14    }
15 }

```

Testcase **Test Result**
Accepted Runtime: 0 ms
 Case 1 Case 2 Case 3

Time Complexity - $O(n^2)$

628. Maximum Product of Three Numbers

```

class Solution {
    public int maximumProduct(int[] nums) {
        Arrays.sort(nums);

        int p1 = (nums[nums.length-1]) * (nums[nums.length
-2]) * (nums[nums.length - 3]);
        int p2 = (nums[0]) * (nums[1]) * (nums[nums.length - 1]);
        int p = p2;
        if (p1 > p2){
            p = p1;
        }

        return p;
    }
}

```

Accepted 93 / 93 testcases passed
Pooja-Rathor submitted at Aug 16, 2026 21:37

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in on...

Runtime: 15 ms Beats 67.47%
Memory: 47.97 MB Beats 58.71%

```
class Solution {
    public int maximumProduct(int[] nums) {
        Arrays.sort(nums);

        int p1 = (nums[nums.length-1])*(nums[nums.length -2])*(nums[nums.length - 3]);
        int p2 = (nums[0])*(nums[1])*(nums[nums.length - 1]);
        int p = p2;
        if (p1 > p2){
            p = p1;
        }

        return p;
    }
}
```

Accepted Runtime: 2 ms
Case 1 Case 2 Case 3

Time Complexity - $O(n \log n)$

26. Remove Duplicates from Sorted Array

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int k = 1;

        for(int i = 1; i < nums.length; i++) {
            if(nums[i] != nums[i - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
}
```

The screenshot displays a LeetCode submission interface. On the left, the problem description and submission status are shown. The submission is 'Accepted' with 362/362 testcases passed, submitted by 'Pooja-Rathor' on Aug 16, 2026 at 21:41. A promotional banner for 'Unlock the Full LeetCode Experience' is visible. Below this, performance metrics are shown: Runtime is 1 ms (Beats 77.40%) and Memory is 46.34 MB (Beats 96.19%). A bar chart illustrates the runtime distribution across different time intervals.

The code editor on the right shows a Java solution for the 'Remove Duplicates from Sorted Array' problem. The solution uses a two-pointer approach to remove duplicates in-place.

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k = 1;
4
5         for(int i = 1; i < nums.length; i++) {
6             if(nums[i] != nums[i - 1]) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11
12        return k;
13    }
14 }
15
```

The 'Testcase' tab at the bottom shows 'Accepted' with a runtime of 0 ms. Two test cases are listed: 'Case 1' and 'Case 2', both of which are passed.

Time Complexity - $O(n)$